

CARTILLA DE APRENDIZAJE · INGENIERÍA INDUSTRIAL

Gestión Inteligente de Inventarios

Módulo didáctico: Ruta guiada para reproducir la simulación, validación y modelado predictivo de inventarios en RStudio.



Módulo didáctico · Ingeniería Industrial

Ruta de aprendizaje del módulo

- 01 Fase 1 · Diseño del entorno de simulación
- 02 Fase 2 · Construcción del dataset sintético
- 03 Fase 3 · Validación del conjunto de datos
- 04 Fase 4 · Preparación de variables predictoras
- 05 Fase 5 · Modelado predictivo y comparación
- 06 Fase 6 · Síntesis y buenas prácticas



Cada fase reproduce, paso a paso, el proceso metodológico desarrollado en la investigación, scripts, datos y decisiones.

Objetivos de aprendizaje y prerequisites

Al finalizar este módulo, el estudiante reproducirá el proceso metodológico completo del proyecto: simulación, validación, preparación de variables y modelado predictivo de la demanda.



Prerrequisitos

R y RStudio a nivel básico, estadística descriptiva, y fundamentos de gestión de inventarios (demanda, lead time, punto de reorden).

¿Qué es la gestión inteligente de inventarios?

Es el uso de datos históricos y simulados de demanda, lead time, costos y niveles de stock junto con modelos estadísticos para anticipar necesidades de reabastecimiento y reducir tanto el sobre stock como el desabastecimiento.

En esta cartilla, el catálogo maestro de productos y sus variables asociadas se usan como caso guía para practicar limpieza de datos, modelado predictivo y visualización en RStudio.



Término clave

Punto de reorden: nivel de stock que dispara una nueva orden de compra antes de agotar existencias.

Flujo de trabajo

1

Diseño del entorno de simulación

Catalogo Maestro

2

Simulación de datos

Carga del dataset simulado

3

Limpieza y preparación

Rezagos, media histórica y estacionalidad

4

Modelado estadístico

Regresión (MLR) y árbol (CART)

5

Validación

MAE, RMSE y WAPE del modelo

6

Interpretación

Resultados y decisiones

Contexto del entorno de Simulación

Una PyME del sector ferretero administra un amplio catálogo de productos, cuyas referencias presentan comportamientos de demanda muy diferentes entre sí. Mientras algunos artículos se venden todos los días, otros registran ventas solo en determinadas épocas o unas pocas veces al año. Esta variabilidad dificulta estimar la demanda mediante métodos tradicionales y, además, la ausencia de datos históricos reales impide desarrollar y evaluar un modelo predictivo de forma confiable.

Solución: Construir un entorno simulado

Se construye un inventario simulado en el que se conocen previamente los parámetros reales del sistema, como la demanda, el lead time y la política de reabastecimiento. Esto permite, en las etapas posteriores, comparar qué tan bien los modelos predictivos logran aproximarse a esa realidad conocida, evaluando su capacidad para representar el comportamiento del sistema con precisión.



Prerrequisitos

R y RStudio a nivel básico, estadística descriptiva, y fundamentos de gestión de inventarios (demanda, lead time, punto de reorden).

¿Qué es el catálogo maestro?

El catálogo maestro es la base sobre la que se construye el inventario simulado. Reúne las 120 referencias (SKU) del estudio y define los atributos estructurales de cada una, como el código, la familia (FER, EPP y HER), la subfamilia y la unidad de medida. A partir de esta información, el simulador asigna posteriormente los parámetros operativos de cada producto, como la rotación esperada, el costo, el margen y el tiempo de entrega (lead time). De esta manera, el catálogo proporciona una estructura organizada y consistente para representar el comportamiento del inventario durante la simulación.

¿De dónde provienen estos datos?

El catálogo maestro no se construyó a partir de una empresa real. Sus características y parámetros fueron definidos como parte del diseño metodológico de la investigación, tomando como referencia la literatura especializada en gestión de inventarios y las condiciones habituales del sector ferretero. Esto permite crear un entorno de simulación realista, controlado y reproducible, ideal para analizar el comportamiento del sistema y evaluar modelos predictivos en diferentes escenarios.



Términos clave

FER: Ferretería

EPP: Elementos de protección Personal

HER: Herramientas

Arquitectura del simulador

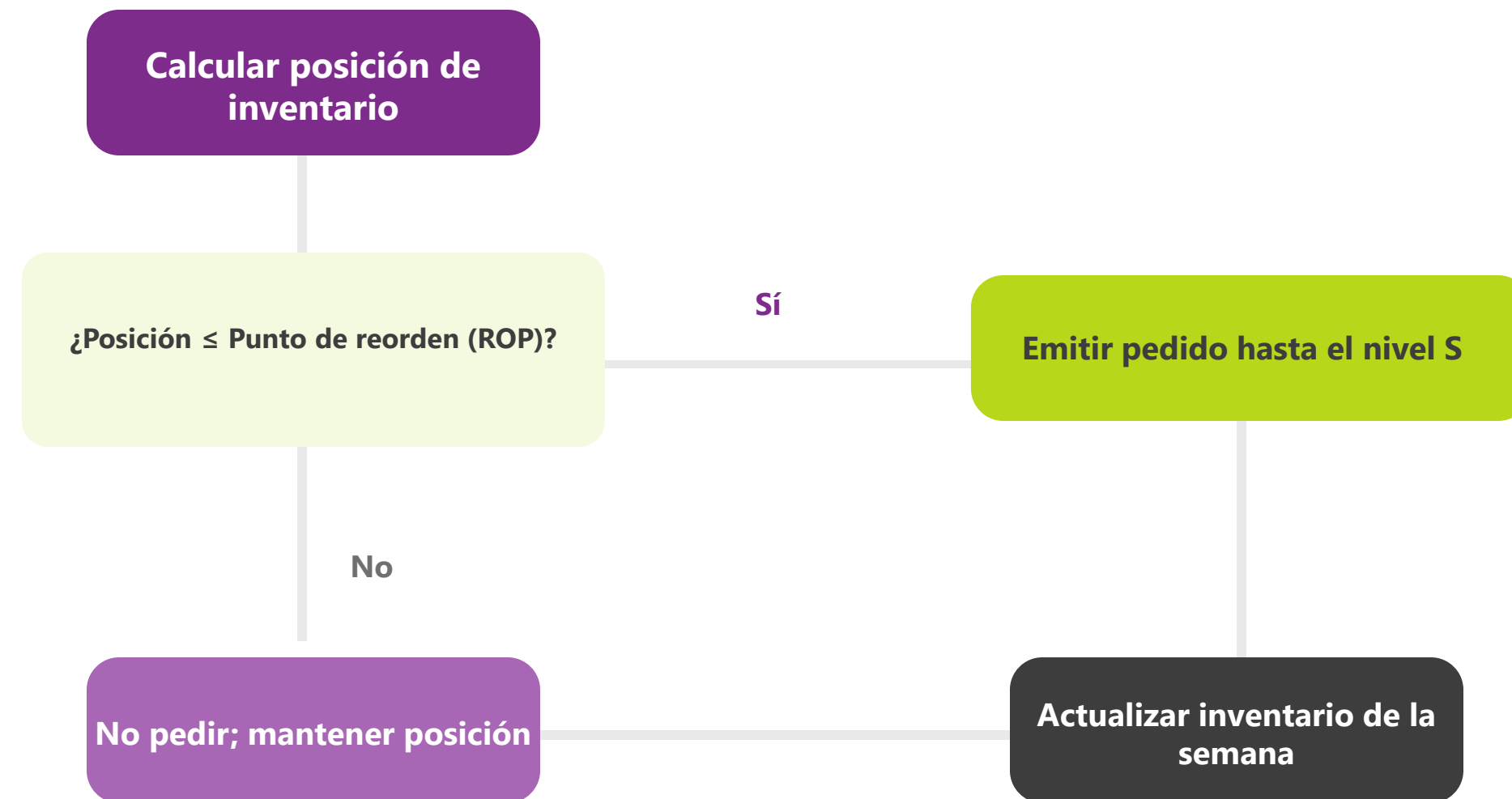


Cada etapa construye sobre la anterior: los atributos estáticos de cada SKU determinan su clasificación ABC, la cual fija los parámetros de la política (s, S) que gobierna la simulación semana a semana.

Catálogo maestro y parámetros

Familia	Distribución (λ)	Lead time	Margen prom.	Amplitud estac.	Costo ordenar
FER	Gamma(4, 15)	Ferretería	0,25 sem	$\approx 40 \%$	20.000 COP
EPP	Gamma(3, 4)	Protección	0,50 sem	$\approx 30 \%$	30.000 COP
HER	Gamma(1.5, 1)	Herramientas	1,00 sem	$\approx 20 \%$	60.000 COP
Política	Revisión periódica	(s, S)	z: 2.05/1.64/1.28	por clase ABC	R=1 semana

Política de inventario (s, S): lógica semanal



Regla de decisión

El sistema revisa cada semana si la posición de inventario (existencias más tránsito) cae al punto de reorden o por debajo; si ocurre, se emite un pedido que repone hasta el nivel objetivo S.

Script: generar_dataset_inventarios.R

```
● ● ●  generar_dataset_inventarios.R

01 set.seed(123)  # semilla oficial
02 lambda_sku <- rgamma(1, shape, scale)
03
04 demanda <- rnbinom(1, size = k, mu = lambda*est)
05 SS <- z_abc[clase] * sigma_LTD
06
07 ROP <- lambda*E_LT + SS
08 if (posicion <= ROP) pedir(hasta = S)
09 write.csv(out, "dataset_inventarios_ferreteria.csv")
```

Cómo leerlo

El generador fija la semilla 123 para asegurar reproducibilidad exacta; cada SKU obtiene sus atributos por muestreo de las distribuciones definidas, y la política (s, S) se recalcula cada semana a partir del punto de reorden (ROP) y el stock de seguridad (SS).

Output esperado: dataset_inventarios_ferreteria.csv



Ejercicio 1 : Justificar las distribuciones del simulador

Para cada componente del simulador (demanda, lead time, margen, costo unitario), identifique la distribución de probabilidad empleada y argumente por qué es más adecuada que una alternativa simple (por ejemplo, Normal o Uniforme).

Pista

La demanda usa Binomial Negativa por su sobredispersión ($\text{Var} > \text{media}$); el lead time usa Gamma por su soporte positivo; el margen usa Beta por estar acotado en $(0,1)$; el costo usa Triangular por aprovechar mínimo, moda y máximo del catálogo.

¿Por qué nunca se debe modelar sin validar?

Un conjunto de datos simulado o real puede parecer correcto y aun así violar las reglas del proceso que lo generó. Modelar sin verificar arriesga construir conclusiones sobre errores silenciosos: fuga de información, inconsistencias en la política de inventario o identidades contables rotas.



Principio metodológico

La validación no se basa en fórmulas genéricas de inventarios, sino en las identidades contables y de política que el propio algoritmo generador impone: solo así se detectan errores reales y no falsos hallazgos.

¿Cómo se validó el conjunto de datos generado?

Una vez generado el conjunto de datos sintéticos, fue necesario verificar que su estructura y contenido fueran consistentes antes de utilizarlo para el modelado predictivo. Para ello se desarrollaron dos scripts complementarios, cada uno con un propósito específico.

Objetivo Script 2: Diagnóstico

Comprender cómo quedó construido el conjunto de datos después de la simulación. Este script permitió conocer su estructura general, verificando aspectos como el número de SKU, el horizonte de tiempo, las familias de productos y el comportamiento de las principales variables

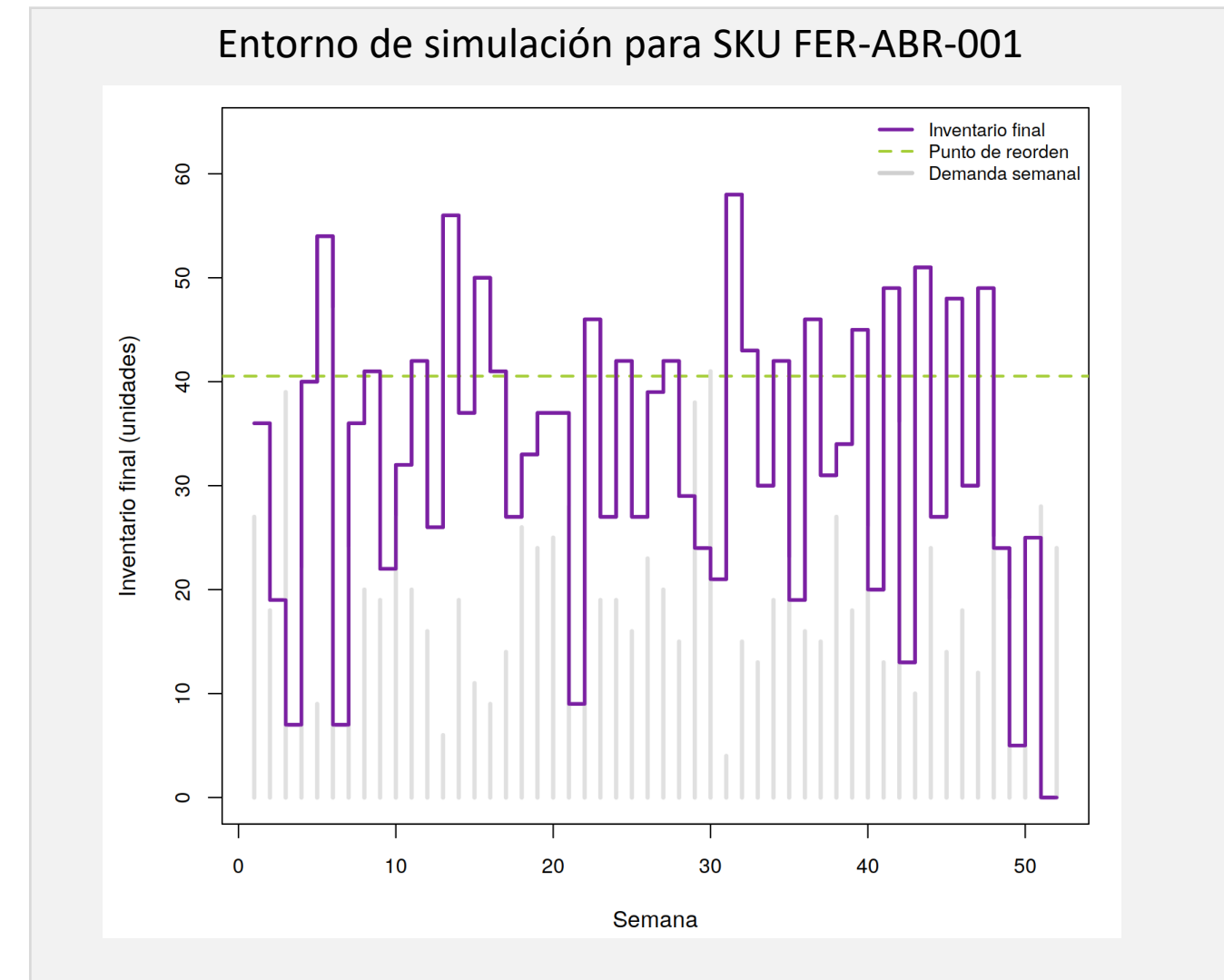
Objetivo Script 3: Validación

Comprobar que toda la información generada cumpliera las reglas establecidas para la simulación. Para ello se ejecutaron 84 verificaciones automáticas, orientadas a identificar posibles inconsistencias, errores de estructura o incumplimientos de las condiciones definidas en el modelo. Esta validación garantiza que el conjunto de datos sea confiable y pueda utilizarse en las siguientes etapas del proyecto.

Diagnóstico del entorno de simulación generado

El entorno reproduce una PyME ferretera con múltiples SKU, generando series de demanda semanal y niveles de stock a lo largo de un ciclo anual.

- 120 SKU simulados en el catálogo maestro
- 52 semanas de historial de demanda (1 año)
- 3 familias con lead times distintos (FER, EPP, HER)
- 6240 Registros
- Archivo CSV



Descripción: Entorno de simulación de inventario para el SKU FER-ABR-001 a lo largo de 52 semanas. La línea morada representa el inventario final de cada semana, que desciende con la demanda (barras grises) y se repone al emitirse un pedido; la línea verde punteada marca el punto de reorden que dispara cada reabastecimiento. El comportamiento reproduce la dinámica real de una política de revisión periódica (s, S).

Distribución de la demanda por familia

La caracterización estadística confirma visualmente la coherencia del dataset: FER concentra los mayores volúmenes de rotación, EPP presenta demanda moderada y HER exhibe un patrón intermitente de bienes de alta inversión.

- Índice de dispersión (Var/media): 8,95 en FER, 3,85 en EPP y 1,39 en HER
- Las tres familias superan 1: justifica el uso de Binomial Negativa sobre Poisson

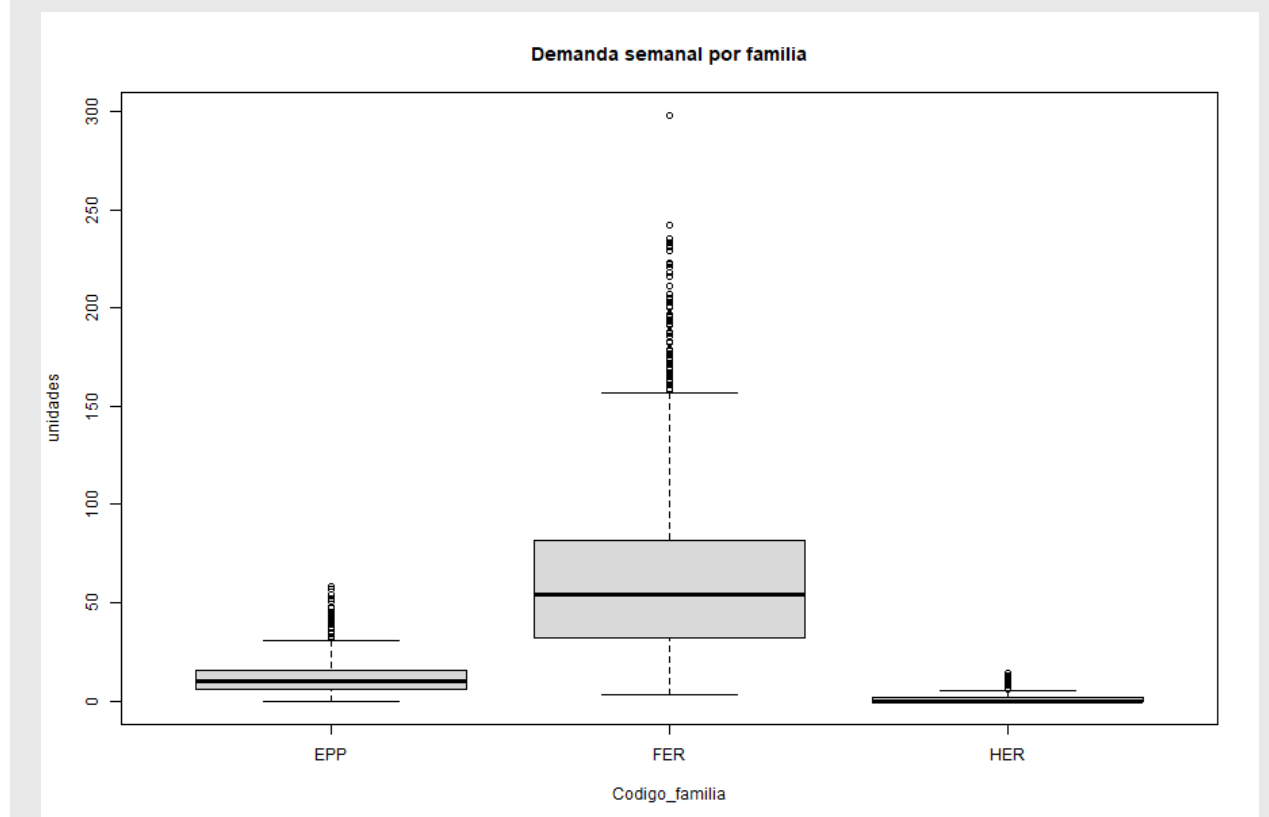


Figura generada en R base

Resultados del Script de validación: 84 validaciones

Dimensión de validación	Verificaciones	Superadas	Críticos
Tipos, dominios y rangos	37	37	0
Política de inventario (s, S)	18	18	0
Identidades contables por fila	10	10	0
Identidades de costos y precios	6	6	0
Integridad, completitud y unicidad	5	5	0
Coherencia temporal y de flujo	3	3	0
Clasificación ABC (Pareto)	2	2	0
Contrastes distribucionales	2	2	0
Valores atípicos	1	1	0
Total	84	84	0

Resultados del Script validador



Ejercicio 2 — Interpretar un hallazgo de validación

Ejecute `validar_dataset.R` sobre el conjunto de datos. Elija tres verificaciones de dimensiones distintas (identidades contables, política de inventario, coherencia temporal) y explique qué error concreto detectarían si fallaran.



Pista

Por ejemplo: si `'inv_inicial(t) = inv_final(t-1)'` fallara, significaría que el inventario no se está transfiriendo correctamente entre semanas consecutivas para ese SKU.

Construcción del modelo predictivo

Una vez preparada la matriz de predictores, comenzó el desarrollo del sistema predictivo. El proceso se organizó en seis bloques, cada uno con una función específica para entrenar, evaluar y comparar los modelos.

Bloque	¿Qué se hizo?	¿Para qué sirvió?
1. Preparación	Se construyó la matriz de predictores y se dividieron los datos en entrenamiento y validación.	Garantizar que los modelos aprendieran únicamente con información histórica.
2. Regresión Lineal Múltiple (MLR)	Se entrenó un modelo de regresión utilizando las variables predictoras más relevantes.	Obtener un modelo base para predecir la demanda semanal.
3. Árbol de Decisión (CART)	Se entrenó un árbol de decisión y posteriormente se simplificó mediante poda.	Construir un modelo capaz de capturar relaciones no lineales entre las variables.
4. Evaluación	Ambos modelos realizaron predicciones sobre datos nunca vistos.	Medir su capacidad de predicción mediante indicadores de error.
5. Comparación	Se compararon los resultados de ambos modelos mediante métricas y pruebas estadísticas.	Identificar cuál modelo ofrecía un mejor desempeño.
6. Visualización	Se generaron gráficos para interpretar y comunicar los resultados obtenidos.	Facilitar el análisis y la explicación de los hallazgos.

Preparación de los datos para el modelado predictivo

Antes de entrenar un modelo predictivo, los datos deben prepararse para que representen únicamente la información que estaría disponible en el momento de realizar una predicción. Este proceso garantiza que el modelo aprenda a anticipar la demanda sin utilizar información del futuro.

¿Qué transformaciones se realizaron? 1. Construcción de variables predictoras

Se generaron nuevas variables a partir del historial de demanda de cada SKU:

- lag_1 a lag_4: demanda de las 4 semanas previas
- Media_historica: promedio expansivo, sin incluir la semana t
- Estacional_det: componente determinística, conocida ex ante

- ventas, inventario_final, quiebre_stock: resultado de la semana t
- demanda_insatisfecha, unidades_recibidas: dependen de la demanda de t
- costo_unitario, precio_venta, margen: no causan la demanda

Nota: Importante: Todas estas variables se construyeron utilizando únicamente información disponible antes de la semana que se desea predecir.

Partición temporal de los datos: 80/20

Una vez construidas las variables predictoras, el conjunto de datos se dividió respetando el orden cronológico.

- **Semanas 1 a 42: entrenamiento del modelo.**
- **Semanas 43 a 52: validación del modelo.**

Esta división evita que el modelo utilice información del futuro para aprender, una condición indispensable en problemas de predicción de series de tiempo.

Predicción one-step-ahead

El modelo predice la demanda de una semana utilizando únicamente la información disponible hasta la semana anterior. Esto evita el uso de datos futuros y hace que la predicción sea realista



Validación de las variables predictoras

Para comprobar que todas las variables fueron calculadas correctamente, se desarrolló un script independiente. Este script reconstruyó cada variable predictora y la comparó con la matriz utilizada para entrenar el modelo, verificando que no existieran errores ni uso de información futura. Resultado de la validación13 verificaciones ejecutadas13 verificaciones superadas Matriz de predictores: **APTA** para el modelado predictivo

Validación de las variables predictoras



Nota: Como las primeras cuatro semanas no disponen de un historial suficiente para calcular los rezagos, fueron excluidas del análisis.
El conjunto de datos pasó de: 6.240 registros → 5.760 registros

Script: Construcción del modelo predictivo

```
matriz_predictores_OE2.R

01 for (k in 1:4)
02   d[[lag_k]] <- ave(demanda, SKU, ...)
03
04 media_hist <- ave((demanda, SKU, FUN = expansiva)
05 dm$conjunto <- ifelse(semana<=42, "train", "valid")
06
07 write.csv(dm, "matriz_modelado_OE2.csv")
08 # excluye ventas, inventario_final, quiebre_stock
09 # validar_predictores_OE2.R certifica el resultado
```

Entrenamiento del modelo MLR

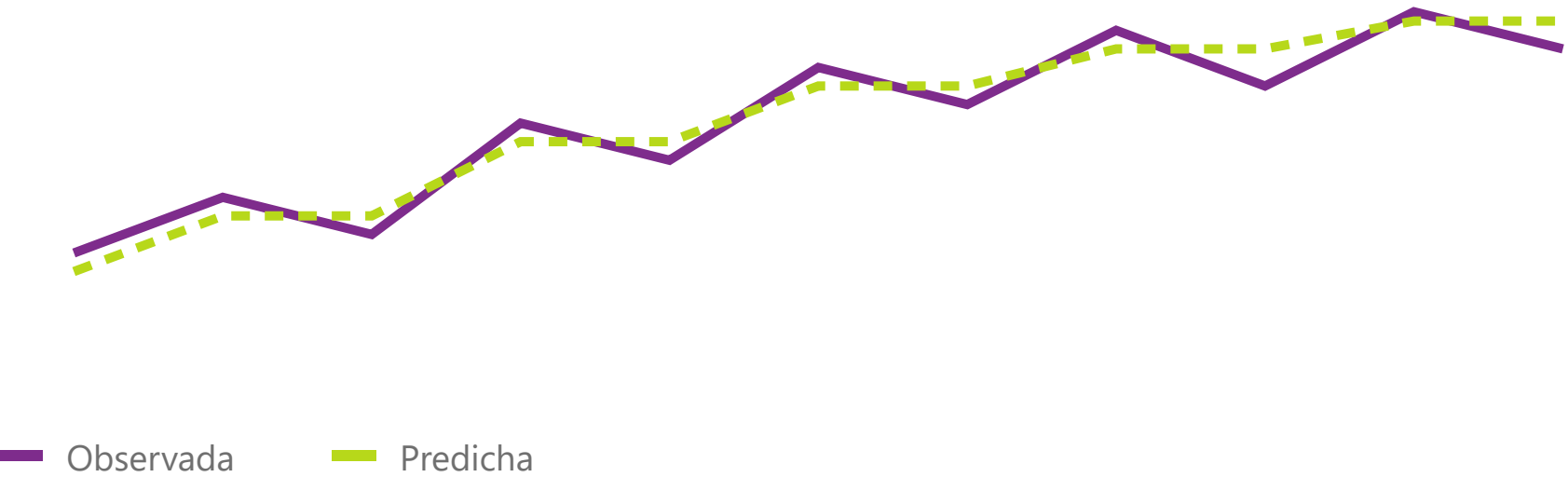
Se entrenó una regresión lineal múltiple utilizando los rezagos, la media histórica y la estacionalidad. También se evaluó si la familia del producto mejoraba el modelo; al no aportar diferencias significativas, fue descartada.

Entrenamiento del modelo CART

Se construyó un árbol de decisión que posteriormente fue podado para conservar únicamente la estructura que ofrecía el mejor equilibrio entre precisión y complejidad.

Resultados del modelo predictivo

Demanda observada vs. predicha (validación)



WAPE
(CART)

32,3%

RMSE
(CART)

21,6 u

Mejora del CART sobre la MLR

+6,7%

Comparación de modelos predictivos

Regresión lineal múltiple (MLR)

- Modelo estadístico paramétrico y de alta interpretabilidad
- Asume linealidad; produce una salida continua
- Implementado con `lm()` de R base

Árbol de decisión — CART

- Algoritmo de aprendizaje automático no paramétrico
- Capta relaciones no lineales; no exige supuestos distribucionales
- Implementado con `rpart()` y poda por validación cruzada

Evaluación comparativa: MLR Vs CART

Métrica	MLR	CART	Mejora CART
MAE (global)	13,91	12,97	+6,7 %
RMSE (global)	21,73	21,60	+0,6 %
WAPE (global)	34,60 %	32,27 %	+6,7 %
Ajuste del modelo	$R^2 = 0,756$	10 hojas	—

Librerías utilizadas en R



R base

Lectura de datos, construcción de variables y regresión con `lm()`, sin dependencias externas.



rpart

Construcción y poda del árbol de decisión para regresión (CART).



graphics (R base)

Figuras de diagnóstico, comparación de modelos y resultados a 300 dpi.

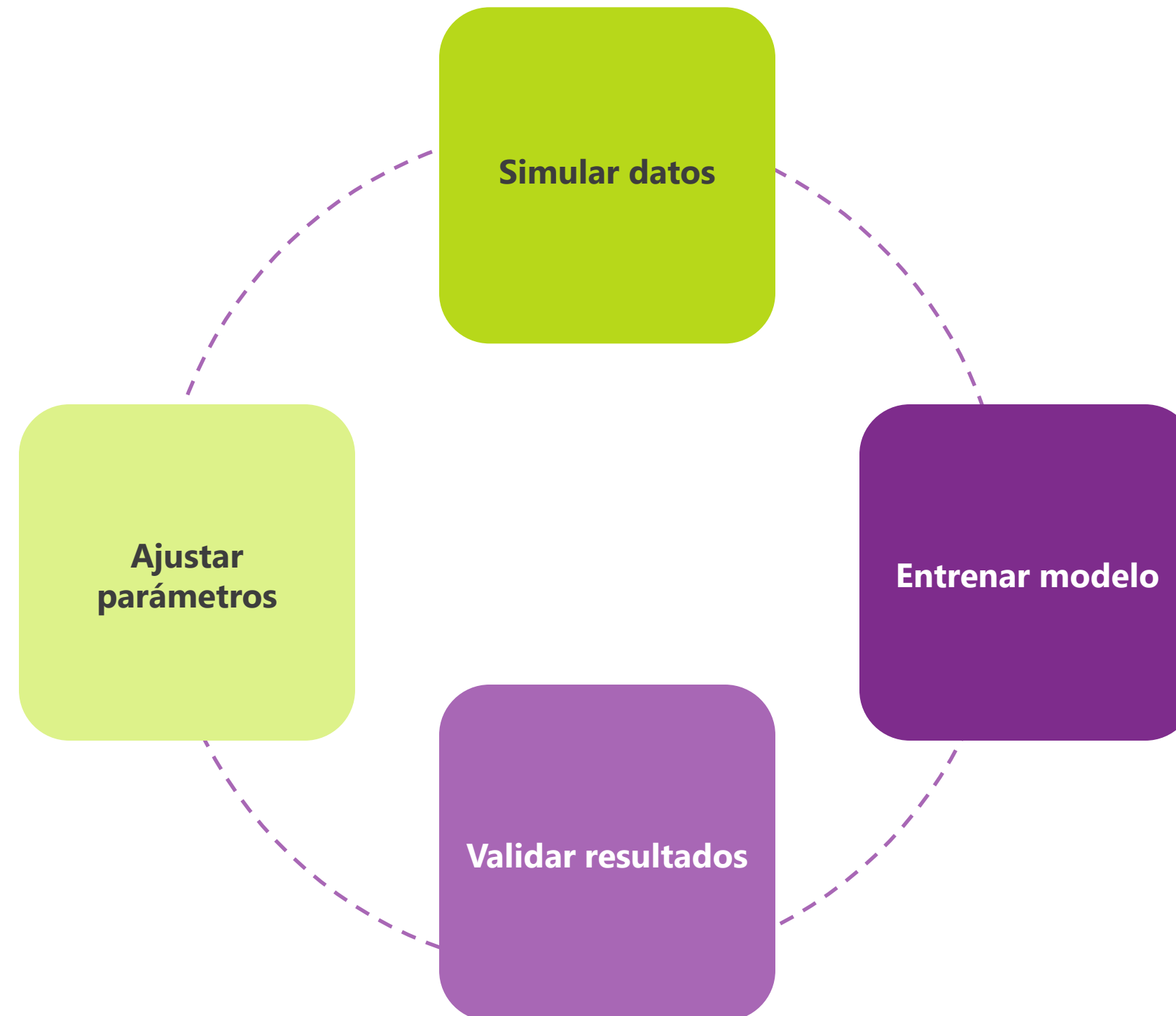


rpart.plot

Visualización del árbol de decisión podado.

Ciclo de mejora del modelo predictivo

El modelo se refina de forma iterativa: cada validación aporta información para ajustar parámetros y volver a entrenar con datos actualizados.



Recomendaciones metodológicas transversales

-  Nunca modele sobre datos sin validar: verifique identidades contables y de política antes de entrenar cualquier modelo.
-  En series de tiempo, particione siempre de forma temporal (hold-out u origen móvil); una partición aleatoria produce fuga de información.
-  Ante demanda intermitente, use WAPE o MASE en vez de MAPE, y compare siempre contra un modelo de referencia (baseline).

Buenas prácticas de reproducibilidad



Semilla fija

`set.seed(123)` al inicio de cada script garantiza que los resultados sean exactamente replicables.



Separación temporal

El conjunto de validación nunca participa en el entrenamiento ni en la construcción de variables.



Validación independiente

Un script separado verifica cada etapa (dataset y predictores) antes de modelar, sin mezclar su lógica con la de construcción.



Repositorio Sistema Predictivo de Inventarios

<https://github.com/mabely2710-dot/sistema-predictivo-inventarios>



Unidades Tecnológicas de Santander

Facultad de Ingeniería · Ingeniería Industrial